

Resumen Ejecutivo: ¿Qué es JEV AI y por qué es crítico para OpenClaw?

JEV AI es un modelo de inteligencia artificial de tipo "**Sistema 1**" (**Reflejos y Decisiones Estructuradas**) desarrollado por **TypeSafe AI** (startup fundada por Diogo Almeida, ex-investigador de OpenAI). A diferencia de los LLMs tradicionales (Claude 3.5 Sonnet, GPT-4o) diseñados para generar prosa conversacional libre, Jev está concebido exclusivamente para **toma de decisiones fuertemente tipadas en milisegundos (< 30ms)** con scores de calibración matemática y costes hasta 200 veces inferiores.

Este informe establece la especificación técnica para que el **Agente de Implementación** integre JEV como la capa de *Sistema Nervioso Autónomo* dentro de OpenClaw Antigravity, eliminando la latencia y el consumo innecesario de tokens en tareas de enrutamiento, guardrails y control de estados.

01 Fundamento Teórico y la Paradoja de Jevons

El diseño de JEV AI se fundamenta en la teoría de los dos sistemas cognitivos de *Daniel Kahneman*:

⚡ Sistema 1: JEV AI (Periférico / Reflejos)

Decisiones binarias, clasificación y enrutamiento instantáneo en menos de 20ms. Salida 100% tipada (Enums, Booleanos, Probabilidades). Sin sobrecarga de tokens ni parsing de texto.

 Sistema 2: Claude / GPT-4o (Córtex / Síntesis)

Razonamiento profundo, redacción de informes médicos/legales, negociación y análisis estratégico. Modelos lentos (2–5 seg) y de alto costo cognitivo.

La Paradoja de Jevons en la IA: Cuando una decisión inteligente pasa de costar 0.03 USD a 0.0001 USD y de tardar 3 segundos a 15 milisegundos, el software no reduce su uso, sino que multiplica por 100 las decisiones automatizadas dentro de los bucles de código de OpenClaw.

02 Arquitectura de Despliegue en OpenClaw Antigravity

En la arquitectura actual de OpenClaw (ej. flujos de MEDISI América, Cuadra Erre, HRB y E-Books), JEV asume el control del tráfico y la validación previa antes de invocar cualquier modelo frontera:

```
[EVENTO ENTRANTE] → (Telegram / WhatsApp / Webhook / Archivo PDF) | ▼
[_____] | ⚡ JEV AI - CAPA DE CONTROL Y TRIAGE
(SISTEMA 1) | | - Enrutamiento por intención (< 20ms) | | - Guardrail de Seguridad & PII
Sanitization | | - Calibrated Confidence Score [0.00 - 1.00] |
[_____] | | (Confidence ≥ 0.85) (Confidence <
0.85) | | ▼▼ [_____] | 🧑 SUBAGENTE DEDICADO | | 🧠
CLAUDE 3.5 / GPT-4o | | - Aria (Regulatorio) | | (Razonamiento Profundo) | | - Staff Comercial | | -
Fallback & Triage | | - Operación OTC v3.4 | | - Auditoría Humana | [_____]
[_____] | ▼ [_____] | ⚡
JEV AI - TRANSICIÓN DE ESTADO FINITO (FSM CHECK) | | ¿Expediente completo? [TRUE / FALSE] →
Obsidian / SQL | [_____]
```


03 Matriz Comparativa de Rendimiento Operativo

Dimensión Operativa	Arquitectura LLM Convencional	Arquitectura Híbrida OpenClaw + JEV	Impacto Clave
Latencia de Triage / Enrutamiento	1,800 – 4,200 ms LENTO	12 – 28 ms INSTANTÁNEO	Reducción de latencia del 99.3%
Costo de Tokens en Control	500 – 1,200 tokens / evento	0 tokens en LLMs frontera	Ahorro del 82% en facturación mensual
Fiabilidad de Salida (Parsing)	JSON frágil / markdown inesperado	Tipado estricto garantizado	Cero excepciones por parsing roto
Detección de PII / Fugas	Post-procesamiento o prompts lentos	Evaluación inline en tiempo real	Cumplimiento normativo LOPD/GDPR nativo
Capacidad de Throughput	~15 peticiones / seg (rate limits)	+1,500 peticiones / seg	Escalabilidad horizontal para PYMES

04 Especificación de Integración para el Agente de Implementación

El Agente de Implementación debe estructurar los módulos de enrutamiento y guardrails utilizando contratos fuertemente tipados. A continuación se define la interfaz maestra para el SDK de OpenClaw:

```
TYPESCRIPT / OPENCLAW AGENT SDK CONTRACT

import { JevClient, DecisionType } from '@openclaw/jev-core';

export interface OpenClawEvent {
  channel: 'telegram' | 'whatsapp' | 'webhook' | 'document_drop';
  payload: string;
  metadata: Record<string, unknown>;
}

export interface TriageDecision {
  targetSubagent: 'aria_regulatory' | 'sales_assistant' | 'board_director' | 'support_queue';
  actionType: 'EXECUTE_TOOL' | 'ESCALATE_SISTEMA_2' | 'REJECT_UNSAFE';
  containsPII: boolean;
  confidenceScore: number; // 0.0 a 1.0 (Calibración exacta)
}

export async function processIncomingEvent(event: OpenClawEvent): Promise<void> {
  // 1. Evaluación instantánea Sistema 1 con JEV
  const decision: TriageDecision = await JevClient.evaluate({
    programState: { channel: event.channel, meta: event.metadata },
    query: event.payload,
    schema: DecisionType.OPENCLAW_TRIAGE_SCHEMA
  });

  // 2. Lógica de Enrutamiento y Fallback Calibrado
  if (decision.confidenceScore < 0.85 || decision.actionType === 'ESCALATE_SISTEMA_2') {
    // Delegar a Claude 3.5 Sonnet / GPT-4o para razonamiento profundo
    return await routeToFrontierLLM(event, decision);
  }
}
```

```
// 3. Ejecución directa a subagente sin sobrecarga de tokens
return await dispatchToSubagent(decision.targetSubagent, event);
}
```

05 Plan de Implementación por Fases (Roadmap)

- **Fase 1 (Día 1–3) – Gateway de Triage:** Reemplazar el prompt de clasificación inicial en los conectores de Telegram/WhatsApp por el endpoint tipado de JEV.
- **Fase 2 (Día 4–6) – Guardrails de Privacidad:** Implementar filtro inline de datos sensibles (PII) antes de cualquier llamada a APIs de terceros.
- **Fase 3 (Día 7–10) – Transición de Estados FSM:** Integrar JEV en los bucles de ejecución del flujo regulatorio OTC v3.4 (MEDISI) y tareas de gestión de Cuadra Erre.
- **Fase 4 (Día 11–14) – Monitoreo y Calibración:** Dashboard de trazabilidad en Mission Control para comparar scores de confianza y ratios de escalamiento a Sistema 2.